

**Автономная некоммерческая профессиональная образовательная организация
«Хекслет колледж»**

КУРСОВОЙ ПРОЕКТ

по дисциплине Основы алгоритмизации и программирования

Тема:

Разработка модуля веб-приложения на стороне клиента

«Бот-калькулятор диет»

Специальность

09.02.07 «Информационные системы и программирование»

Руководитель

Г. А. Королёв

подпись

И.О.Фамилия

Студент

Протасовский Е.А.

Группа

02.24.ИСИП.ОФ.9-
НСК

Фамилия И.О.

Работа выполнена с оценкой _____

Санкт-Петербург – 2025 год

1. ВВЕДЕНИЕ

Определение темы

В современном мире вопросы здорового питания и контроля массы тела становятся всё более актуальными. Для эффективного управления рационом необходимо знать индивидуальную норму калорий, белков, жиров и углеводов (далее — КБЖУ). Существует множество методик расчёта, однако большинство из них требуют ручных вычислений или использования громоздких электронных таблиц. Разработка специализированного веб-приложения для автоматизации этого процесса является востребованной задачей.

Актуальность проекта

Актуальность данной темы обусловлена ростом популярности здорового образа жизни и фитнеса, что приводит к увеличению числа людей, следящих за своим питанием. Существующие аналоги (MyFitnessPal, FatSecret, Яндекс.Здоровье) имеют существенные недостатки: платную подписку, необходимость регистрации, отсутствие функции автоматической генерации меню или сложный интерфейс. Разрабатываемое приложение является бесплатным, не требует установки и работает в любом современном браузере, что делает его доступным для широкой аудитории.

Цель проекта

Целью данного проекта является разработка интерактивной информационной системы — веб-приложения «Бот-калькулятор диет», которое позволяет подбирать персонализированный рацион питания на основе целей пользователя (похудение, набор массы, поддержание веса) и его физиологических параметров.

Задачи проекта

Для достижения поставленной цели были определены следующие задачи:

1. Провести анализ предметной области и существующих аналогов.
2. Сформулировать функциональные и нефункциональные требования к разрабатываемой системе.
3. Спроектировать архитектуру приложения и структуры данных.
4. Реализовать модуль расчёта КБЖУ с использованием формулы Миффлина-Сан Жеора.
5. Разработать модуль генерации меню на основе базы рецептов.
6. Реализовать сохранение и загрузку профиля пользователя с использованием localStorage.
7. Создать пользовательский интерфейс с валидацией ввода.
8. Выполнить тестирование и отладку приложения.

Объект и предмет исследования

Объектом исследования является процесс проектирования и разработки клиентских веб-приложений с использованием инструментальных средств JavaScript.

Предметом исследования выступает создание системы для расчёта КБЖУ и генерации меню, включающее разработку объектной модели данных (классы User, Recipe),

реализацию алгоритмов расчёта и подбора блюд, построение пользовательского интерфейса и организацию локального хранения с помощью технологии localStorage.

Информационная база

При выполнении проекта использовались учебники и учебные пособия по программированию, официальная документация MDN Web Docs, ECMAScript спецификации, ресурсы сети Интернет для разработчиков (Stack Overflow, GitHub, Habr), а также инструментальные средства разработки: VS Code, система контроля версий Git, браузерные инструменты разработчика (DevTools).

Структура проекта

Курсовой проект состоит из введения, основной части, заключения, списка источников информации и приложений.

Первая глава содержит описание предметной области, анализ существующих аналогов веб-приложений для расчёта калорий, формулировку функциональных и нефункциональных требований к разрабатываемой системе, а также обоснование выбора технологий и инструментов разработки.

Вторая глава посвящена проектированию архитектуры приложения, разработке объектной модели данных, реализации модуля расчёта КБЖУ, модуля генерации меню, модуля работы с локальным хранилищем и созданию пользовательского интерфейса.

Третья глава описывает процесс интеграции всех модулей в единое веб-приложение, проведение тестирования и анализ полученных результатов.

В приложениях представлены листинги ключевых модулей и скриншоты пользовательского интерфейса.

2. ОСНОВНАЯ ЧАСТЬ

2.1 Структурно-семантический анализ пользовательского интерфейса (HTML5)

Интерфейс программного продукта сверстан в соответствии со спецификацией HTML5. Архитектура графического интерфейса строится на модульном принципе, где каждый элемент выполняет строго определённую функцию.

Глобальным контейнером выступает блочный элемент `<div class="container">`, который центрирует содержимое и изолирует всю логику интерфейса. Внутреннее пространство контейнера разбито на несколько функциональных блоков.

Блок верхней панели содержит кнопку загрузки профиля с идентификатором `loadTopBtn`, предназначенную для восстановления ранее сохранённых данных пользователя из локального хранилища. Данное архитектурное решение позволяет пользователю продолжить работу с приложением после перезагрузки страницы без необходимости повторного ввода параметров.

Заголовочный блок (header) включает текстовый заголовок первого уровня `<h1>` с наименованием продукта «DietCalc» и подзаголовок `<p class="subtitle">` с кратким описанием функционала.

Вычислительный модуль (форма с идентификатором `dietForm`) содержит:

- Поля ввода числовых параметров: возраст (`#age`), вес (`#weight`), рост (`#height`). Для обеспечения мобильной доступности полям присвоен атрибут `inputmode="numeric"`, активирующий цифровую клавиатуру на сенсорных устройствах. Стилистическое оформление полей включает подавление стандартных стрелочных контролов браузера через CSS-правила `-moz-appearance: textfield` и `::-webkit-inner-spin-button`.
- Раскрывающийся список выбора пола (`#gender`) с опциями «Женский» и «Мужской».
- Раскрывающийся список цели (`#goal`) с тремя опциями: «Похудение», «Поддержание веса», «Набор массы».
- Раскрывающийся список уровня физической активности (`#activity`) с опциями: «Низкая», «Средняя», «Высокая».
- Интерактивную кнопку отправки формы с типом `submit`, активирующую расчётный модуль.

Модуль отображения результатов (блок с идентификатором `result`) изначально скрыт CSS-классом `hidden`. После выполнения расчёта в данном блоке динамически отображаются вычисленные значения калорий, белков, жиров и углеводов. Каждый показатель размещён в отдельном элементе `<div class="kbju-grid">` с уникальными идентификаторами (`#calories`, `#protein`, `#fat`, `#carbs`).

Модуль генерации меню содержит кнопку с идентификатором `dayMenuBtn`, активирующую подсистему подбора блюд. Сгенерированное меню отображается в блоке `#menuBlock`, который включает контейнер для контента `#menuContent` и кнопки обновления (`#refreshMenuBtn`) и закрытия (`#closeMenuBtn`).

Модуль управления профилем включает кнопку сохранения (`#saveBtn`), которая сериализует текущие параметры пользователя и результаты расчёта в локальное хранилище.

Блок системных сообщений (`#message`) предназначен для информирования пользователя о статусе операций (сохранение, загрузка, ошибки валидации). Сообщения автоматически деактивируются через три секунды после отображения.

2.2 Организация модульной архитектуры и объектной модели

Программный комплекс построен по принципу разделения ответственности (Model-View-Controller, далее — MVC). Данная архитектурная парадигма обеспечивает высокую читаемость, масштабируемость и удобство поддержки кода. Каждый модуль размещён в отдельном файле и подключается в HTML-документе в строго определённом порядке.

Таблица 1 — Структура модульной архитектуры

Компонент	Файл	Функциональное назначение
-----------	------	---------------------------

Модель данных	calculator.js	Реализация формулы Миффлина-Сан Жеора, объектная модель User, расчётные коэффициенты
Модель хранения	storage.js	Интерфейс взаимодействия с Web Storage API (localStorage)
Модель генерации	menuGenerator.js	База рецептов, объектная модель Recipe, алгоритм подбора блюд
Представление	ui.js	Рендеринг результатов, отображение меню, системные сообщения
Контроллер	app.js	Инициализация, регистрация обработчиков событий, связывание компонентов

Объектная модель данных. В соответствии с требованиями объектно-ориентированного программирования, в системе определены классы User и Recipe.

Листинг 1 — Класс User (объектная модель пользователя)

javascript

```
class User {
  constructor(age, weight, height, gender, activity, goal) {
    this.age = age;           // возраст в годах
    this.weight = weight;     // вес в килограммах
    this.height = height;     // рост в сантиметрах
    this.gender = gender;     // пол ('male' или 'female')
    this.activity = activity; // уровень активности
    this.goal = goal;         // цель ('weight_loss', 'maintain',
    'muscle_gain')
  }
}
```

Листинг 2 — Класс Recipe (объектная модель рецепта)

javascript

```
class Recipe {
  constructor(id, name, type, calories, proteins, fats, carbs,
  ingredients, instruction) {
    this.id = id;              // уникальный идентификатор
    this.name = name;          // наименование блюда
    this.type = type;          // тип приёма пищи
    this.calories = calories;  // калорийность в ккал
    this.proteins = proteins;  // содержание белков в граммах
    this.fats = fats;          // содержание жиров в граммах
    this.carbs = carbs;        // содержание углеводов в граммах
  }
}
```

```

        this.ingredients = ingredients; // список ингредиентов
        this.instruction = instruction; // пошаговый рецепт
    }
}

```

Внедрение объектной модели позволяет стандартизировать структуру данных, упростить передачу параметров между модулями и повысить надёжность типизации в среде разработки.

2.3 Реализация математического ядра расчёта КБЖУ

Для расчёта базального уровня метаболизма (Basal Metabolic Rate, далее — BMR) в программном продукте используется формула Миффлина-Сан Жеора, разработанная в 1990 году и признанная мировым диетологическим сообществом как наиболее точная для современного человека.

Математическое выражение BMR имеет следующий вид:

- Для мужчин:

$$\text{BMR} = 10 \times \text{вес(кг)} + 6.25 \times \text{рост(см)} - 5 \times \text{возраст(лет)} + 5$$
- Для женщин:

$$\text{BMR} = 10 \times \text{вес(кг)} + 6.25 \times \text{рост(см)} - 5 \times \text{возраст(лет)} - 161$$

Листинг 3 — Функция расчёта базального метаболизма

javascript

```

function calculateBMR(age, weight, height, gender) {
    if (gender === "male") {
        return 10 * weight + 6.25 * height - 5 * age + 5;
    } else {
        return 10 * weight + 6.25 * height - 5 * age - 161;
    }
}

```

Коэффициенты физической активности. Для учёта уровня физической нагрузки введены следующие мультипликативные коэффициенты:

Таблица 2 — Коэффициенты физической активности

Уровень активности	Описание	Коэффициент
Низкая (low)	Сидячая работа, отсутствие спортивных нагрузок	1.2
Средняя (moderate)	1-3 тренировки в неделю	1.375
Высокая (high)	3-5 тренировок в неделю	1.55

Итоговая суточная калорийность с учётом активности вычисляется по формуле:

Калории(активность) = BMR × КоэффициентАктивности

Коэффициенты целевой коррекции. Для достижения поставленной пользователем цели применяются следующие корректирующие коэффициенты:

Таблица 3 — Коэффициенты целевой коррекции

Цель	Коэффициент	Физиологическое обоснование
Похудение (weight_loss)	0.85	Создание дефицита калорий (15%)
Поддержание веса (maintain)	1.00	Энергетический баланс
Набор массы (muscle_gain)	1.15	Создание профицита калорий (15%)

Итоговая суточная калорийность:
Калории(финал) = Калории(активность) × КоэффициентЦели

Расчёт макронутриентов (БЖУ). Распределение калорийности по макронутриентам зависит от выбранной цели:

Таблица 4 — Распределение макронутриентов по целям

Нутриент	Похудение	Поддержание	Набор массы
Белки (%)	35%	25%	30%
Жиры (%)	25%	30%	25%
Углеводы (%)	40%	45%	45%

Пересчёт процентного соотношения в граммы осуществляется по энергетическим эквивалентам:

- 1 грамм белка = 4 килокалории
- 1 грамм жира = 9 килокалорий
- 1 грамм углеводов = 4 килокалории

2.4 Разработка модуля генерации меню и базы рецептов

Для автоматического подбора рациона питания разработан модуль генерации меню, включающий базу из 20 рецептов, распределённых по четырём типам приёмов пищи.

Структура базы рецептов:

Таблица 5 — Распределение рецептов по типам приёмов пищи

Тип приёма	Идентификатор	Количество рецептов	Примеры блюд
Завтрак	breakfast	5	Овсяная каша, омлет, творог с фруктами
Обед	lunch	5	Куриная грудка с гречкой, суп-пюре из брокколи
Ужин	dinner	5	Салат Цезарь, запечённый лосось, овощное рагу
Перекус	snack	5	Греческий йогурт, протеиновый коктейль

Каждый рецепт содержит следующие атрибуты: уникальный идентификатор, наименование блюда, тип приёма пищи, калорийность, содержание белков, жиров и углеводов, размер порции, список ингредиентов, пошаговая инструкция приготовления.

Алгоритм генерации меню. При активации пользователем кнопки «Меню на день» запускается вычислительный процесс, состоящий из следующих этапов:

1. Определение целевой калорийности из ранее рассчитанных данных (глобальная переменная `lastKBJUData`).
2. Итеративный перебор всех типов приёма пищи (завтрак → обед → ужин → перекус).
3. Фильтрация базы рецептов по текущему типу.
4. Случайный выбор одного рецепта из отфильтрованного массива.
5. Накопление суммарной калорийности выбранных блюд.
6. Вычисление процентного отклонения итоговой калорийности от целевой нормы.

Листинг 4 — Функция генерации меню

javascript

```
function generateDayMenu(kbju) {
    const menu = {};

    mealOrder.forEach(function(type) {
        const filteredRecipes = recipes.filter(function(recipe) {
            return recipe.type === type;
        });
        menu[type] = filteredRecipes.length > 0
            ? filteredRecipes[Math.floor(Math.random() *
filteredRecipes.length)]
            : null;
    });

    let totalCalories = 0;
    const menuValues = Object.values(menu);
    for (let i = 0; i < menuValues.length; i++) {
        const recipe = menuValues[i];
        if (recipe !== null) {
            totalCalories = totalCalories + recipe.calories;
        }
    }

    const targetCalories = kbju.calories;
    const diffPercent = Math.abs(totalCalories - targetCalories) /
targetCalories * 100;

    return {
        menu: menu,
```



```

        total: totalCalories,
        target: targetCalories,
        diff: diffPercent
    };
}

```

2.5 Организация локального хранения данных (localStorage)

Для обеспечения долгосрочного хранения профиля пользователя между сессиями в программном продукте реализована подсистема персистентного хранения на базе Web Storage API (localStorage).

Технические характеристики механизма localStorage:

- Объём хранилища: не менее 5 мегабайт на источник.
- Время жизни: данные сохраняются до явного удаления пользователем или скриптом.
- Доступность: данные изолированы по источнику (протокол + домен + порт).
- Интерфейс: синхронный (без промисов).

Структура сохраняемого профиля. Сериализуемый объект профиля имеет следующую структуру:

javascript

```

{
  user: {
    age: 30,
    weight: 70,
    height: 170,
    gender: "male",
    goal: "maintain",
    activity: "moderate"
  },
  kbju: {
    calories: 2224,
    protein: 139,
    fat: 74,
    carbs: 250
  },
  savedAt: "18.06.2026, 14:30:25"
}

```

Листинг 5 — Модуль работы с localStorage

javascript

```

function saveProfile(user, kbju) {
  try {
    localStorage.setItem("dietProfile", JSON.stringify({

```

```

        user: user,
        kbj: kbj,
        savedAt: new Date().toLocaleString()
    ));
    return true;
} catch(error) {
    return false;
}
}

function loadProfile() {
    try {
        const saved = localStorage.getItem("dietProfile");
        return saved ? JSON.parse(saved) : null;
    } catch(error) {
        return null;
    }
}

```

Обработка ошибок. Модуль хранения включает механизм обработки исключительных ситуаций (конструкция `try...catch`). При возникновении ошибки сериализации или десериализации данных пользователь информируется через систему сообщений, а выполнение операции прерывается без негативного воздействия на работу основного приложения.

3. ЗАКЛЮЧЕНИЕ

Часть 1. Выводы по теоретической части

В ходе выполнения теоретической части курсового проекта было установлено, что проблема персонализированного подбора рациона питания является актуальной. Проведённый анализ существующих аналогов (MyFitnessPal, FatSecret, Яндекс.Здоровье) выявил их недостатки: платную подписку, отсутствие функции генерации меню, необходимость регистрации. Разработанное техническое задание определило полный перечень функциональных и нефункциональных требований к системе.

Часть 2. Результаты по задачам

1. **Проведён анализ предметной области** — изучены основные понятия КБЖУ, проанализированы 3 аналога, выявлены их преимущества и недостатки.
2. **Сформулировано техническое задание** — определены функциональные и нефункциональные требования, обоснован выбор технологий.
3. **Спроектирована архитектура** — разработана модульная структура из 5 файлов, созданы классы User и Recipe.
4. **Реализован модуль расчёта КБЖУ** — используется формула Миффлина-Сан Жеора, учтены все параметры.

5. **Разработана база рецептов** — 20 рецептов с КБЖУ, ингредиентами и инструкцией приготовления.
6. **Реализована генерация меню** — подбор блюд на завтрак, обед, ужин, перекус с отображением отклонения от нормы.
7. **Создан пользовательский интерфейс** — чёрно-белый дизайн, адаптивная вёрстка, валидация ввода.
8. **Реализовано локальное хранение** — сохранение и загрузка профиля через localStorage.
9. **Выполнено тестирование** — 6 сценариев, все пройдены успешно.

Общий вывод

Разработанное веб-приложение «Бот-калькулятор диет» полностью соответствует техническому заданию и может использоваться для персонализированного расчёта нормы КБЖУ и генерации меню. Приложение является бесплатным, не требует установки и регистрации, работает в любом современном браузере.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32–2001. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — Введ. 2002–07–01. — Минск : Межгосударственный совет по стандартизации, метрологии и сертификации ; Москва : Изд-во стандартов, 2001. — 20 с.
2. Федеральный государственный образовательный стандарт среднего профессионального образования по специальности 09.02.07 «Информационные системы и программирование» (утв. Приказом Минобрнауки РФ от 09.12.2016 №1547).
3. Документация по веб-технологиям MDN Web Docs : справочник для веб-разработчиков. — URL: <https://developer.mozilla.org> (дата обращения: 18.06.2026). — Режим доступа: свободный. — Текст : электронный.
4. Веб-платформа Doka Guide : открытый справочник по фронтенд-разработке и программированию. — URL: <https://doka.guide> (дата обращения: 18.06.2026). — Режим доступа: свободный. — Текст : электронный.
5. Сервис хостинга IT-проектов GitHub : облачная платформа для управления кодом и совместной разработки. — URL: <https://github.com> (дата обращения: 18.06.2026). — Режим доступа: свободный. — Текст : электронный.
6. Миффлин М.Д., Сан Жеор С.Т. Новая predictive equation для расчёта resting energy expenditure // Американский журнал клинического питания. — 1990. — Т. 51, № 2. — С. 241–247.
7. Web Storage API — официальная документация. — URL: https://developer.mozilla.org/ru/docs/Web/API/Web_Storage_API (дата обращения: 18.06.2026). — Режим доступа: свободный. — Текст : электронный.
8. ECMAScript 2024 Language Specification (ECMA-262). — URL: <https://262.ecma-international.org> (дата обращения: 18.06.2026). — Режим доступа: свободный. — Текст : электронный.